Machine Learning — Challenge 4
Learning from Demonstration with Adversarial Training (`GAIL` on Atari) — Three-Way Comparison with Challenges 1 & 3

Prof. Carlos Andrés Sierra, M.Sc.
Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

Overview

This document describes Challenge 4 for the Machine Learning course. It is the third and final extension of Challenge 1, completing a three-algorithm comparison series:

Challenge 1 (`DQN`) ⟶ Challenge 3 (`PPO`) ⟶ Challenge 4 (`GAIL`: Learning from Demonstration + Adversarial Training)

Each group continues working on the **same `ALE` environment** assigned in Challenge 1. The new learning paradigm introduced here is **Generative Adversarial Imitation Learning (`GAIL`)** (Ho & Ermon, 2016): a framework that combines ideas from `Generative Adversarial` Networks (`GANs`) and Reinforcement Learning to allow an agent to learn from *demonstrations* without requiring an explicit reward function from the environment.

The central research question of Challenge 4 is:

> *"Does an agent that learns by imitating demonstrations — through adversarial training rather than direct reward maximisation — converge faster, transfer better, or reach higher final performance than pure RL agents (`DQN`, `PPO`) on the assigned Atari game? Under which conditions does the source of demonstrations matter?"*

Demonstration source. A central practical decision students must make is where the demonstrations come from. Three legitimate sources are authorised in this challenge:

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Full-time Adjunct Professor at Universidad Distrital Francisco José de Caldas.

Any comment or concern about this document can be sent to: *cavirguezs@udistrital.edu.co*.

1. **Self-generated**: Roll out the best DQN checkpoint from Challenge 1 (or PPO from Challenge 3) and record (s_t, a_t) tuples. The expert is imperfect, which is realistic and scientifically interesting.
2. **Policy-gradient warm-start**: Train PPO for a short budget (e.g., 500 000 steps) and use those trajectories as demonstrations; then train GAIL with the adversarial signal replacing the environment reward.
3. **Pre-collected Atari demonstrations** (optional, if available): Use publicly available human demonstration datasets (e.g., Atari Grand Challenge dataset) for a small number of games.

Regardless of the source, the quality and quantity of demonstrations must be clearly documented and treated as a variable in the experimental analysis.

Challenge objective

Groups will:

1. Implement a **Behavioral Cloning (BC)** baseline that directly minimises the cross-entropy between demonstrations and policy outputs. This establishes a supervised-learning lower bound.
2. Implement a **GAIL agent**: a discriminator network trained adversarially against the policy, providing a learned reward signal used with PPO as the inner RL algorithm.
3. Collect and document a demonstration dataset from the group's own challenge agents (or from an authorised external source).
4. Conduct experiments over the demonstration quantity and quality, and compare GAIL against BC, DQN (Challenge 1), and PPO (Challenge 3) using the same evaluation protocol.
5. Produce a scientific report in IEEE format extending the Challenge 1 and Challenge 3 papers with GAIL results and a three-way algorithmic comparison.

Environments — same assignment as Challenges 1 and 3

Each group works on the **same ALE game** assigned in Challenge 1. Preprocessing must be **identical** to Challenges 1 and 3 (grayscale, 84×84, frame-stack 4, frame-skip 4, pixels in $[0, 1]$) so results are directly comparable across all three challenges.

1. Group 1 — **ALE/MontezumaRevenge-v5**
2. Group 2 — **ALE/Pitfall-v5**
3. Group 3 — **ALE/PrivateEye-v5**
4. Group 4 — **ALE/Gravitar-v5**
5. Group 5 — **ALE/Solaris-v5**

6. Group 6 — **ALE/Venture-v5**
7. Group 7 — **ALE/MsPacman-v5**
8. Group 8 — **ALE/Phoenix-v5**

Theoretical background

Behavioral Cloning (BC)

BC treats imitation as a supervised learning problem. Given a demonstration dataset $\mathcal{D} = \{(s_i, a_i)\}_{i=1}^{N}$, BC minimises the negative log-likelihood:

$$\mathcal{L}_{\mathrm{BC}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \pi_\theta(a_i \mid s_i) \tag{1}$$

BC is fast to train but suffers from *distributional shift*: the policy's own errors accumulate at test time because the learnt distribution does not cover states visited only by the agent, not the demonstrator.

Generative Adversarial Imitation Learning (GAIL)

GAIL frames imitation as a two-player game between a **policy** π_θ (the generator) and a **discriminator** D_ϕ. The discriminator is trained to distinguish expert state-action pairs from policy-generated ones:

$$\max_{D_\phi} \; \mathbb{E}_{(s,a)\sim\mathcal{D}}[\log D_\phi(s,a)] + \mathbb{E}_{(s,a)\sim\pi_\theta}[\log(1 - D_\phi(s,a))] \tag{2}$$

The policy is then trained with any RL algorithm — here PPO — using the *adversarial reward*:

$$r_{\mathrm{adv}}(s,a) = -\log(1 - D_\phi(s,a)) \quad \text{or equivalently} \quad r_{\mathrm{adv}}(s,a) = \log D_\phi(s,a) \tag{3}$$

This signal encourages the policy to produce trajectories that the discriminator cannot separate from the expert demonstrations, driving the learnt policy towards the expert's occupancy measure without requiring the environment's true reward. The full GAIL objective is:

$$\min_{\pi_\theta} \; \max_{D_\phi} \; \mathbb{E}_{(s,a)\sim\pi_\theta}[\log D_\phi(s,a)] + \mathbb{E}_{(s,a)\sim\mathcal{D}}[\log(1 - D_\phi(s,a))] - \lambda H(\pi_\theta) \tag{4}$$

where $H(\pi_\theta)$ is the policy entropy regulariser (same as PPO's entropy bonus).
Training alternates between:

- **Discriminator step**: update D_ϕ with a mini-batch of expert and agent transitions using binary cross-entropy.
- **Policy step**: run a PPO update using r_{adv} as the reward signal (the environment reward is *not* used).

Required implementation elements

- **Demonstration collector**: a script that loads a trained checkpoint (DQN from Challenge 1 or PPO from Challenge 3) and records a configurable number of (s_t, a_t) pairs into a file (e.g., .npz or pickle).
- **Behavioral Cloning baseline**: a training loop that minimises $\mathcal{L}_{\mathrm{BC}}$, followed by direct evaluation (no further RL).
- **Discriminator network**: a CNN that takes a stacked-frame observation (and optionally the action) as input and outputs a scalar in $(0, 1)$.
- **GAIL training loop**: alternating discriminator updates (binary cross-entropy) and PPO updates (using adversarial reward).
- **Reward replacement**: the environment's true reward must be completely replaced by r_{adv} during GAIL training. The true reward is used only for evaluation.
- **Demonstration ablation**: at least two demonstration dataset sizes (e.g., 5 000 and 50 000 state-action pairs) must be tested to study the effect of demonstration quality/quantity.

Suggested hyperparameter search space for GAIL

- Discriminator learning rate: 1×10^{-4}; 3×10^{-4}; 5×10^{-4}.
- Discriminator update frequency: 1 update per PPO rollout; 5 updates per PPO rollout.
- Discriminator architecture: shared CNN backbone (frozen or jointly trained); separate CNN.
- Demonstration dataset size: 5 000; 20 000; 50 000 (s, a) pairs.
- Demonstration quality: from best DQN checkpoint vs. from mid-training DQN checkpoint (deliberately imperfect).
- PPO hyperparameters: inherit the best configuration found in Challenge 3 (no need to re-sweep these; treat them as fixed unless investigation motivates otherwise).
- Entropy coefficient λ: 0.001; 0.01; 0.02.
- Gradient penalty coefficient (optional, for training stability): 0.0; 10.0.

Three-way comparison methodology

The following protocol must be applied so that DQN, PPO, and GAIL results are directly comparable. All three algorithms must be evaluated under the same conditions.

1. **Budget parity**: fix a total environment step budget (e.g., 5 000 000). For GAIL, environment steps count only the steps the *agent* takes; discriminator gradient updates do not count.

2. **Identical preprocessing and evaluation**: same wrappers, same 10-episode deterministic evaluation at fixed intervals.

3. **Metrics to report for all three algorithms**:

   - Learning curve: episode return vs. environment steps.
   - Sample efficiency: steps to reach the target score threshold defined in Challenge 3.
   - Final performance: mean $\pm$ std over 3 seeds at end of training.
   - Training stability: AUC normalised by total steps.
   - For GAIL additionally: discriminator accuracy over training time (does it collapse? does it stay informative?).

4. **BC baseline**: evaluated without any RL steps; treated as a zero-step proxy for demonstration quality.

5. **Analysis questions** students must address:

   - Does GAIL outperform pure RL (DQN, PPO) on games with sparse rewards? If yes, why? If no, why not?
   - How sensitive is GAIL to demonstration quality and quantity?
   - Does the adversarial reward signal remain informative throughout training, or does the discriminator collapse?
   - In what regime is BC alone competitive with full GAIL or PPO?

Shared implementation: demonstration collector, BC, and GAIL core

The preprocessing wrapper and AtariActorCritic backbone from Challenge 3 apply unchanged. The following additional components are required.

Listing 1: Demonstration collection from a saved checkpoint

```python
import numpy as np
import torch
import gymnasium as gym
# Reuse make_env and AtariActorCritic from Challenge 3

def collect_demonstrations(env_id: str, checkpoint_path: str,
                           n_steps: int = 20_000, seed: int = 0,
                           device: str = "cpu") -> dict:
    """
    Roll out a saved policy and record (obs, action) pairs.

    Returns a dict with keys 'observations' and 'actions',
    each a numpy array of shape (n_steps, ...).
    """
    env = make_env(env_id, seed=seed)
    n_actions = env.action_space.n

    model = AtariActorCritic(n_actions).to(device)
    model.load_state_dict(torch.load(checkpoint_path, map_location=device
        ))
```

```
    model.eval()

    obs_buf, act_buf = [], []
    obs, _ = env.reset()
    for _ in range(n_steps):
        obs_t = torch.tensor(obs, dtype=torch.float32).unsqueeze(0).to(
            device)
        with torch.no_grad():
            logits, _ = model(obs_t)
        action = logits.argmax(dim=-1).item()   # greedy / deterministic

        obs_buf.append(obs)
        act_buf.append(action)

        obs, _, terminated, truncated, _ = env.step(action)
        if terminated or truncated:
            obs, _ = env.reset()

    env.close()
    demos = {
        "observations": np.array(obs_buf, dtype=np.float32),
        "actions":      np.array(act_buf, dtype=np.int64),
    }
    np.savez_compressed("demos.npz", **demos)
    print(f"Saved {n_steps} demo steps to demos.npz")
    return demos
```

Listing 2: Behavioral Cloning (BC) baseline

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, TensorDataset

def train_bc(env_id: str, demos_path: str = "demos.npz",
             n_epochs: int = 20, batch_size: int = 256,
             lr: float = 1e-4, device: str = "cpu"):
    """
    Supervised imitation: minimise cross-entropy between
    demonstrations and policy logits.
    """
    data = np.load(demos_path)
    obs_t = torch.tensor(data["observations"], dtype=torch.float32)
    act_t = torch.tensor(data["actions"],      dtype=torch.long)
    dataset = TensorDataset(obs_t, act_t)
    loader  = DataLoader(dataset, batch_size=batch_size, shuffle=True)

    env = make_env(env_id)
    n_actions = env.action_space.n
    env.close()

    model = AtariActorCritic(n_actions).to(device)
    optimizer = optim.Adam(model.parameters(), lr=lr)
    criterion = nn.CrossEntropyLoss()

    for epoch in range(n_epochs):
```

```
        total_loss = 0.0
        for obs_b, act_b in loader:
            obs_b, act_b = obs_b.to(device), act_b.to(device)
            logits, _ = model(obs_b)
            loss = criterion(logits, act_b)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            total_loss += loss.item()
        avg = total_loss / len(loader)
        print(f"BC epoch {epoch+1}/{n_epochs}  loss={avg:.4f}")

    torch.save(model.state_dict(), "bc_policy.pt")
    return model
```

Listing 3: Discriminator network for GAIL

```
import torch
import torch.nn as nn

class GAILDiscriminator(nn.Module):
    """
    Takes a stacked-frame observation (and optionally a one-hot encoded
    action) and outputs P(expert | s, a) in (0, 1).

    Using observation only (obs-only variant) is simpler and often
    sufficient for image-based environments.
    """
    def __init__(self, n_actions: int, use_action: bool = False):
        super().__init__()
        self.use_action = use_action
        # Shared CNN - same architecture as the policy backbone
        self.cnn = nn.Sequential(
            nn.Conv2d(4, 32, kernel_size=8, stride=4), nn.ReLU(),
            nn.Conv2d(32, 64, kernel_size=4, stride=2), nn.ReLU(),
            nn.Conv2d(64, 64, kernel_size=3, stride=1), nn.ReLU(),
            nn.Flatten(),
        )
        cnn_out = 64 * 7 * 7   # 3136
        fc_in   = cnn_out + n_actions if use_action else cnn_out

        self.fc = nn.Sequential(
            nn.Linear(fc_in, 512), nn.Tanh(),
            nn.Linear(512,   1),   nn.Sigmoid(),
        )

    def forward(self, obs, actions_onehot=None):
        feats = self.cnn(obs)
        if self.use_action and actions_onehot is not None:
            feats = torch.cat([feats, actions_onehot], dim=-1)
        return self.fc(feats).squeeze(-1)
```

Listing 4: GAIL training loop (PPO as inner RL algorithm)

```
import torch
```

```python
import torch.nn.functional as F
import torch.optim as optim
from torch.distributions import Categorical

def train_gail(env_id, demos_path="demos.npz",
               total_steps=5_000_000, horizon=1024,
               n_ppo_epochs=4, batch_size=128,
               lr_policy=2.5e-4, lr_disc=3e-4,
               disc_updates_per_rollout=5,
               gamma=0.99, gae_lambda=0.95,
               clip_eps=0.2, ent_coef=0.01, vf_coef=0.5,
               max_grad_norm=0.5, seed=42,
               device="cuda" if torch.cuda.is_available() else "cpu"):

    # --- load demonstrations ---
    data     = np.load(demos_path)
    demo_obs = torch.tensor(data["observations"], dtype=torch.float32)
    demo_act = torch.tensor(data["actions"],      dtype=torch.long)
    n_demos  = len(demo_obs)

    env       = make_env(env_id, seed=seed)
    n_actions = env.action_space.n

    policy = AtariActorCritic(n_actions).to(device)
    disc   = GAILDiscriminator(n_actions, use_action=False).to(device)

    opt_policy = optim.Adam(policy.parameters(), lr=lr_policy)
    opt_disc   = optim.Adam(disc.parameters(),   lr=lr_disc)
    bce        = torch.nn.BCELoss()

    obs, _      = env.reset()
    ep_return   = 0.0
    all_returns = []

    for global_step in range(0, total_steps, horizon):

        # ---- rollout collection ----
        obs_buf, act_buf, logp_buf = [], [], []
        rew_buf, done_buf, val_buf = [], [], []

        for _ in range(horizon):
            obs_t = torch.tensor(obs, dtype=torch.float32).unsqueeze(0).to(device)
            with torch.no_grad():
                logits, value = policy(obs_t)
            dist   = Categorical(logits=logits)
            action = dist.sample()

            obs_buf.append(obs_t.squeeze(0))
            act_buf.append(action)
            logp_buf.append(dist.log_prob(action))
            val_buf.append(value.squeeze())

            obs, env_reward, terminated, truncated, _ = env.step(action.item())
```

```python
            done = terminated or truncated
            done_buf.append(done)
            ep_return += env_reward

            if done:
                all_returns.append(ep_return)
                ep_return = 0.0
                obs, _ = env.reset()

        # ---- adversarial reward (replace env reward) ----
        obs_stack    = torch.stack(obs_buf).to(device)
        with torch.no_grad():
            d_scores = disc(obs_stack)          # P(expert | s)
        # reward: log D(s,a)  -- agent wants to look like the expert
        adv_rewards = torch.log(d_scores + 1e-8).cpu()
        rew_buf = adv_rewards.tolist()

        # ---- GAE advantages ----
        with torch.no_grad():
            obs_t    = torch.tensor(obs, dtype=torch.float32).unsqueeze
                (0).to(device)
            _, nv    = policy(obs_t)
        advantages, returns = compute_gae(
            rew_buf, val_buf, done_buf, nv.item(), gamma, gae_lambda
        )

        # ---- discriminator update ----
        act_one_hot = F.one_hot(
            torch.stack(act_buf), n_actions).float().to(device)

        for _ in range(disc_updates_per_rollout):
            # sample expert mini-batch
            idx_e  = torch.randint(0, n_demos, (batch_size,))
            e_obs  = demo_obs[idx_e].to(device)
            # agent mini-batch
            idx_a  = torch.randint(0, horizon, (batch_size,))
            a_obs  = obs_stack[idx_a]

            d_expert = disc(e_obs)
            d_agent  = disc(a_obs)

            loss_disc = bce(d_expert, torch.ones_like(d_expert)) + \
                        bce(d_agent,  torch.zeros_like(d_agent))
            opt_disc.zero_grad()
            loss_disc.backward()
            opt_disc.step()

        # ---- PPO update ----
        act_t  = torch.stack(act_buf).to(device)
        logp_t = torch.stack(logp_buf).detach().to(device)
        adv_t  = (advantages - advantages.mean()) / (advantages.std() + 1
            e-8)
        ret_t  = returns.to(device)

        idx = torch.randperm(horizon)
```

```
        for _ in range(n_ppo_epochs):
            for start in range(0, horizon, batch_size):
                mb     = idx[start:start + batch_size]
                lg, vn = policy(obs_stack[mb])
                dn     = Categorical(logits=lg)
                lp_new = dn.log_prob(act_t[mb])
                ent    = dn.entropy().mean()
                ratio  = (lp_new - logp_t[mb]).exp()

                s1 = ratio * adv_t[mb]
                s2 = ratio.clamp(1-clip_eps, 1+clip_eps) * adv_t[mb]
                l_pi = -torch.min(s1, s2).mean()
                l_vf = ((vn - ret_t[mb]) ** 2).mean()
                loss  = l_pi + vf_coef * l_vf - ent_coef * ent

                opt_policy.zero_grad()
                loss.backward()
                torch.nn.utils.clip_grad_norm_(policy.parameters(),
                    max_grad_norm)
                opt_policy.step()

        if len(all_returns) % 10 == 0 and all_returns:
            mean_ret = np.mean(all_returns[-100:])
            d_acc    = ((d_expert > 0.5).float().mean() +
                        (d_agent  < 0.5).float().mean()) / 2
            print(f"step={global_step}  ret={mean_ret:.1f}  "
                  f"disc_loss={loss_disc.item():.3f}  "
                  f"disc_acc={d_acc.item():.2f}")

    env.close()
    return policy, disc, all_returns
```

Note on `compute_gae`: reuse the same helper defined (or referenced) in Challenge 3. The only change is that `rew_buf` now contains adversarial rewards instead of environment rewards.

Per-game starter guides

All groups share the common code above. The starters below specify the recommended initial configuration, the source of demonstrations, and the game-specific hypothesis students should investigate.

Group 1 — ALE/MontezumaRevenge-v5

Hypothesis. `DQN` and `PPO` both score near 0 due to extreme reward sparsity. GAIL bypasses the reward entirely — if the demonstration policy visits Room 1, the adversarial reward will implicitly guide the agent there even without a game score signal. Students should test whether GAIL is the first of the three algorithms to consistently enter Room 1.

Listing 5: Group 1 — Montezuma's Revenge GAIL starter

```
# Step 1: collect 50,000 steps from the best \texttt{DQN} checkpoint
collect_demonstrations(
    env_id          = "ALE/MontezumaRevenge-v5",
```

```
    checkpoint_path = "challenge1/group1/best_dqn.pt",
    n_steps         = 50_000,
)

# Step 2: BC baseline (how far does supervised cloning get alone?)
train_bc(
    env_id     = "ALE/MontezumaRevenge-v5",
    demos_path = "demos.npz",
    n_epochs   = 30,
)

# Step 3: \texttt{GAIL} - high entropy coefficient for residual exploration
policy, disc, returns = train_gail(
    env_id                    = "ALE/MontezumaRevenge-v5",
    demos_path                = "demos.npz",
    total_steps               = 5_000_000,
    horizon                   = 2048,
    disc_updates_per_rollout  = 5,
    ent_coef                  = 0.02,  # entropy crucial: demos are sparse
    seed                      = 42,
)
# Ablation: try demos from mid-training \texttt{DQN} (worse quality) vs best.
# Key metric: first step at which agent enters Room 1.
```

Group 2 — ALE/Pitfall-v5

Hypothesis. Even an imperfect DQN demonstrator that survives a few seconds provides a meaningful occupancy measure. GAIL should learn to avoid the worst traps faster than PPO because the adversarial reward penalises states never visited by the demonstrator.

Listing 6: Group 2 — Pitfall! GAIL starter

```
collect_demonstrations(
    env_id          = "ALE/Pitfall-v5",
    checkpoint_path = "challenge1/group2/best_dqn.pt",
    n_steps         = 20_000,   # Pitfall demos are short - collect less
)

train_bc(env_id="ALE/Pitfall-v5", n_epochs=20)

policy, disc, returns = train_gail(
    env_id                    = "ALE/Pitfall-v5",
    total_steps               = 5_000_000,
    horizon                   = 1024,
    disc_updates_per_rollout  = 3,
    ent_coef                  = 0.01,
    gamma                     = 0.995,
    seed                      = 42,
)
# Ablation: 5,000 demos vs 20,000 demos.
# Measure: minimum 'negative reward episodes' per 100 training episodes.
```

Group 3 — ALE/PrivateEye-v5

Hypothesis. Private Eye is so difficult that even the DQN demonstrator likely scores 0. This group should study the *failure case* of GAIL: if demonstrations are uninformative (zero-score trajectories), can the adversarial signal still provide useful guidance? Compare BC, GAIL, and PPO all scoring 0 and reason about why.

Listing 7: Group 3 — Private Eye GAIL starter

```
collect_demonstrations(
    env_id          = "ALE/PrivateEye-v5",
    checkpoint_path = "challenge1/group3/best_dqn.pt",
    n_steps         = 50_000,
)

train_bc(env_id="ALE/PrivateEye-v5", n_epochs=20)

policy, disc, returns = train_gail(
    env_id                   = "ALE/PrivateEye-v5",
    total_steps              = 5_000_000,
    horizon                  = 2048,
    disc_updates_per_rollout = 5,
    ent_coef                 = 0.02,
    seed                     = 42,
)
# KEY experiment: plot discriminator accuracy over time.
# If D collapses to 0.5, the signal is uninformative -- document this.
# Compare discriminator accuracy across all three games as a meta-
    analysis.
```

Group 4 — ALE/Gravitar-v5

Hypothesis. Gravitar requires maintaining specific thrust sequences. GAIL may learn the *style* of control (e.g., constant small thrusts) from the discriminator before the agent discovers any reward. Test whether GAIL's BC warm-start helps stabilise early training compared to PPO from scratch.

Listing 8: Group 4 — Gravitar GAIL starter

```
collect_demonstrations(
    env_id          = "ALE/Gravitar-v5",
    checkpoint_path = "challenge1/group4/best_dqn.pt",
    n_steps         = 30_000,
)

# Initialise GAIL policy with BC weights for a warm-start
bc_model = train_bc(env_id="ALE/Gravitar-v5", n_epochs=25)

policy, disc, returns = train_gail(
    env_id                   = "ALE/Gravitar-v5",
    total_steps              = 5_000_000,
    horizon                  = 1024,
    disc_updates_per_rollout = 3,
    ent_coef                 = 0.01,
```

```
    seed                        = 42,
)
# Ablation: GAIL with BC warm-start vs GAIL from random init.
# Metric: action histogram -- does the agent learn thrust patterns?
```

Group 5 — ALE/Solaris-v5

Hypothesis. Multi-stage games may benefit most from imitation because demonstrations implicitly encode when to switch strategy. GAIL's occupancy-measure matching should learn stage-transition behaviour better than epsilon-greedy exploration. Students should count how many distinct in-game stages each algorithm reaches.

Listing 9: Group 5 — Solaris GAIL starter

```
collect_demonstrations(
    env_id          = "ALE/Solaris-v5",
    checkpoint_path = "challenge1/group5/best_dqn.pt",
    n_steps         = 50_000,
)

train_bc(env_id="ALE/Solaris-v5", n_epochs=25)

policy, disc, returns = train_gail(
    env_id                    = "ALE/Solaris-v5",
    total_steps               = 5_000_000,
    horizon                   = 2048,
    disc_updates_per_rollout  = 5,
    gamma                     = 0.995,
    gae_lambda                = 0.97,
    ent_coef                  = 0.01,
    seed                      = 42,
)
# Track: maximum in-game stage reached per episode.
# Compare with DQN and PPO on this metric.
```

Group 6 — ALE/Venture-v5

Hypothesis. Venture's heavy penalty for taking damage may cause the GAIL agent to over-imitate the demonstrator's conservative behaviour and never explore new dungeons. Students should test whether increasing the entropy bonus or reducing demonstrations quality forces GAIL to deviate from the demonstrator and find more reward.

Listing 10: Group 6 — Venture GAIL starter

```
collect_demonstrations(
    env_id          = "ALE/Venture-v5",
    checkpoint_path = "challenge1/group6/best_dqn.pt",
    n_steps         = 20_000,
)

train_bc(env_id="ALE/Venture-v5", n_epochs=20)

policy, disc, returns = train_gail(
```

```
    env_id                    = "ALE/Venture-v5",
    total_steps               = 5_000_000,
    horizon                   = 1024,
    disc_updates_per_rollout  = 3,
    clip_eps                  = 0.1,
    ent_coef                  = 0.02,
    seed                      = 42,
)
# Ablation: high-quality demos (best DQN) vs low-quality (early DQN checkpoint).
# Question: does lower-quality demo lead to more dungeon exploration?
```

Group 7 — ALE/MsPacman-v5

Hypothesis. Ms. Pac-Man has relatively dense rewards. Here GAIL may converge *more slowly* than PPO or DQN because the adversarial reward is a proxy rather than the true game signal. Students should study whether BC alone — which can imitate good movement patterns — is competitive with full PPO or GAIL on this denser-reward game.

Listing 11: Group 7 — Ms. Pac-Man GAIL starter

```
collect_demonstrations(
    env_id          = "ALE/MsPacman-v5",
    checkpoint_path = "challenge1/group7/best_dqn.pt",
    n_steps         = 50_000,
)

# BC is expected to perform well here -- evaluate it rigorously
bc_model = train_bc(env_id="ALE/MsPacman-v5", n_epochs=30, lr=1e-4)

policy, disc, returns = train_gail(
    env_id                    = "ALE/MsPacman-v5",
    total_steps               = 5_000_000,
    horizon                   = 1024,
    disc_updates_per_rollout  = 3,
    ent_coef                  = 0.01,
    seed                      = 42,
)
# PRIMARY COMPARISON: BC score vs GAIL vs PPO vs DQN on this game.
# This is the richest four-way comparison in the challenge series.
```

Group 8 — ALE/Phoenix-v5

Hypothesis. Phoenix requires fast reactions. A discriminator that evaluates single frames (no action context) may struggle to capture reactive timing. Students should compare a *state-only* discriminator (`use_action=False`) with a *state-action* discriminator (`use_action=True`) to test whether including action context helps the adversarial reward signal.

Listing 12: Group 8 — Phoenix GAIL starter

```
collect_demonstrations(
    env_id          = "ALE/Phoenix-v5",
```

```
    checkpoint_path = "challenge1/group8/best_dqn.pt",
    n_steps         = 30_000,
)

train_bc(env_id="ALE/Phoenix-v5", n_epochs=20)

# Ablation A: state-only discriminator
policy_a, disc_a, ret_a = train_gail(
    env_id      = "ALE/Phoenix-v5",
    horizon     = 512,
    n_ppo_epochs= 10,
    seed        = 42,
)

# Ablation B: modify GAILDiscriminator(use_action=True) and retrain
# compare learning curves of A vs B to assess action-context benefit
```

Deliverables

- **Repository folder:** Add `challenge4/group<k>/` to the same GitHub repository. Include all GAIL source code, the demonstration collection script, the BC training script, a `README.md` with exact run instructions, and logging artifacts for all three algorithms (`DQN`, `PPO`, `GAIL`).
- **Extended IEEE paper:** Extend the Challenge 1 (and 3) paper to include GAIL results and the full three-way comparison. The paper is limited to 10 pages (excluding references). Submit as `challenge4_group<k>_paper.pdf`.
- **Demonstration dataset metadata:** Include a short plain-text or JSON file (`demos_info.txt`) describing the source checkpoint used, the number of steps collected, and the mean/std return of the demonstrating policy.
- **Checklist:** A `CHECKLIST.md` inside the repository folder with:
  - Exact commands to collect demonstrations, train `BC`, and train `GAIL`.
  - Seeds used for all repeated experiments.
  - Pointers to logs and figures for `DQN`, `PPO`, and `GAIL`.
  - A 200-word comparative summary: under what conditions did GAIL add value over pure RL on this specific environment?

Evaluation criteria

- **Implementation correctness (25%):** `BC` baseline is correct; discriminator and `GAIL` loop are properly implemented; adversarial reward replaces environment reward during training.
- **Experimental rigour (25%):** demonstration ablation is performed (at least two dataset sizes), variance is reported over 3 seeds, and all three algorithms use identical evaluation conditions.

- **Comparison quality and analysis (35%):** the three-way comparison (DQN, PPO, GAIL) is fair, the four analysis questions are addressed with empirical evidence, and the discriminator dynamics are discussed.
- **Presentation and writing (15%):** quality of the extended IEEE paper (updated figures, tables, related work on imitation learning, and updated conclusions covering all three challenges).

Notes on scope and computational budget

GAIL requires evaluating the discriminator on both expert and agent batches at every rollout, which adds modest overhead. Discriminator gradient updates are fast but accumulate over training. If compute is limited:

- Reduce `total_steps` to 2 000 000 and document this constraint.
- Use a frozen CNN for the discriminator (only train the fully-connected head) to reduce memory and compute.
- Prioritise the 3-seed DQN vs PPO vs GAIL comparison over wide hyperparameter sweeps.

References and further reading

Core references for this challenge: Ho & Ermon (2016, GAIL), Fu et al. (2018, AIRL), Ross et al. (2011, DAgger), Goodfellow et al. (2014, GANs), Schulman et al. (2017, PPO), and Mnih et al. (2015, DQN). Students should additionally consult recent survey articles on imitation learning and offline RL. All citations must be in IEEE style.

Challenge 4 closes the three-algorithm arc of this course. DQN explored the value-based paradigm; PPO introduced on-policy actor-critic methods; GAIL asks whether an agent can learn to behave well without ever being told what "well" means via a numerical reward. Answering this question rigorously — not just empirically but conceptually — is the goal of this final challenge.